34. Python: Audio Processing

Listen. Analyze. Transform.

Previous Section:

</> 33. Python: PyGame

Contents:

Audio is everywhere. From music streaming services and podcasts to voice assistants and speech recognition systems, modern applications rely heavily on audio data. Unlike images, audio is a time-based signal that contains information about frequency, amplitude, rhythm, and countless hidden patterns waiting to be discovered.

Fortunately, Python provides a rich ecosystem of libraries that make audio processing surprisingly accessible. Whether you want to analyze music, detect

speech characteristics, modify recordings, or build intelligent audio applications, Python offers tools for every stage of the process.

There are many libraries available for audio processing, but two stand out because of their simplicity, power, and popularity:

- **Librosa** – Designed for audio analysis and feature extraction.
- **PyDub** – Designed for audio manipulation and editing.

Think of them this way: **Librosa** helps you *understand* audio. **PyDub** helps you *modify* audio. Together, they form an excellent toolkit for anyone interested in audio processing with Python.

1. Introduction to Librosa and PyDub

Before processing audio, it is important to understand the strengths of each library.

Feature	Librosa	PyDub
Primary Purpose	Audio Analysis	Audio Editing
Best For	Music Information Retrieval, Feature Extraction	Cutting, Merging, Converting Audio
Input Formats	WAV, MP3, OGG, FLAC and more	WAV, MP3, OGG, FLAC and more
Visualization Support	Yes	Limited
Signal Processing	Extensive	Basic
Audio Effects	Limited	Extensive
Machine Learning Applications	Excellent	Minimal
Learning Curve	Moderate	Easy

1.1. Installing the Libraries

Both libraries can be installed using pip:

```
pip install librosa
pip install pydub
```

For certain audio formats such as MP3, PyDub also relies on FFmpeg:

```
brew install ffmpeg
# or
sudo apt install ffmpeg
```

1.2. Librosa – Understanding Audio Signals

Librosa is one of the most widely used Python libraries for audio analysis. Instead of treating an audio file as simply something we listen to, Librosa treats it as data that can be measured, analyzed, and transformed.

With Librosa, we can:

- Load audio files
- Visualize waveforms
- Analyze frequencies
- Extract musical information
- Compute spectrograms
- Detect beats and tempo
- Extract machine learning features

This makes Librosa particularly useful in: Music analysis; Speech processing; Audio classification; Sound recognition; Machine Learning projects

Common Librosa Functions

Function	Purpose
`librosa.load()`	Load an audio file
`librosa.get_duration()`	Calculate audio duration
`librosa.stft()`	Short-Time Fourier Transform
`librosa.istft()`	Inverse STFT
`librosa.amplitude_to_db()`	Convert amplitude to decibels
`librosa.feature.mfcc()`	Extract MFCC features
`librosa.feature.chroma_stft()`	Extract chroma features

Function	Purpose
`librosa.feature.spectral_centroid()`	Measure spectral center
`librosa.beat.beat_track()`	Detect tempo and beats
`librosa.effects.time_stretch()`	Change playback speed
`librosa.effects.pitch_shift()`	Shift audio pitch
`librosa.display.waveshow()`	Display waveform
`librosa.display.specshow()`	Display spectrogram

In short, Librosa focuses on answering questions such as: What frequencies exist in this audio? What is the tempo of this song? What features can describe this sound?

1.2. PyDub – Manipulating Audio Files

While Librosa specializes in analysis, PyDub focuses on practical audio editing. Its interface is simple and intuitive, making common editing tasks only a few lines of code away.

With PyDub, we can:

- Trim audio clips
- Merge recordings
- Split files
- Change volume
- Apply fade effects
- Convert formats
- Export processed audio

This makes PyDub ideal for: Podcast editing; Audio preprocessing; Automated audio workflows; Media conversion; Sound effect generation

Common PyDub Functions

Function	Purpose
`AudioSegment.from_file()`	Load audio
`AudioSegment.export()`	Save audio

Function	Purpose
`AudioSegment.set_frame_rate()`	Change sample rate
`AudioSegment.set_channels()`	Change channel count
`AudioSegment.fade_in()`	Apply fade-in effect
`AudioSegment.fade_out()`	Apply fade-out effect
`AudioSegment.reverse()`	Reverse audio
`AudioSegment.speedup()`	Increase playback speed
`AudioSegment.overlay()`	Mix audio tracks
`AudioSegment.append()`	Concatenate audio
`AudioSegment.split_to_mono()`	Split stereo channels
`AudioSegment.apply_gain()`	Adjust volume
`AudioSegment.silent()`	Generate silence
`AudioSegment.normalize()`	Normalize volume

PyDub focuses on practical questions such as: ***How can I trim this recording? How can I combine multiple audio files? How can I increase the volume or add effects?***

1.4. Librosa vs PyDub

A useful way to remember the difference is:

Task	Recommended Library
Detect beats	Librosa
Extract MFCCs	Librosa
Build ML audio features	Librosa
Visualize waveforms	Librosa
Cut audio clips	PyDub
Merge recordings	PyDub
Convert formats	PyDub
Add fade effects	PyDub
Adjust volume	PyDub

In practice, many real-world projects use both libraries together:

1. Use **PyDub** to prepare and clean audio files.
2. Use **Librosa** to analyze and extract features.
3. Feed those features into Machine Learning models or visualization tools.

And that is why Librosa and PyDub are often considered one of the most powerful combinations for audio processing in Python.

2. Audio Processing with Librosa and PyDub

Now that we understand the roles of Librosa and PyDub, it's time to use them for actual audio processing.

In this section, we will learn how to:

- Load and play audio files
- Extract audio metadata
- Visualize waveforms
- Generate synthetic sounds
- Combine multiple audio signals
- Adjust volume levels
- Apply audio filters
- Export processed audio
- And even implement a simple TTS system

Together, these operations form the foundation of modern audio analysis and manipulation.

2.1. Playing and Analyzing Audio

The first step in audio processing is loading an audio file and understanding its properties.

Using PyDub, we can easily play audio files and convert between formats.
Using Librosa, we can inspect the underlying signal and visualize its waveform.

In this example, we:

1. Convert an MP3 file into WAV format.
2. Play the audio.
3. Extract metadata such as:
    - Sample Rate
    - Duration
    - File Size
    - Amplitude Range
4. Visualize the waveform.

Note: PyDub requires FFmpeg for MP3 support.

bird_singing.mp3

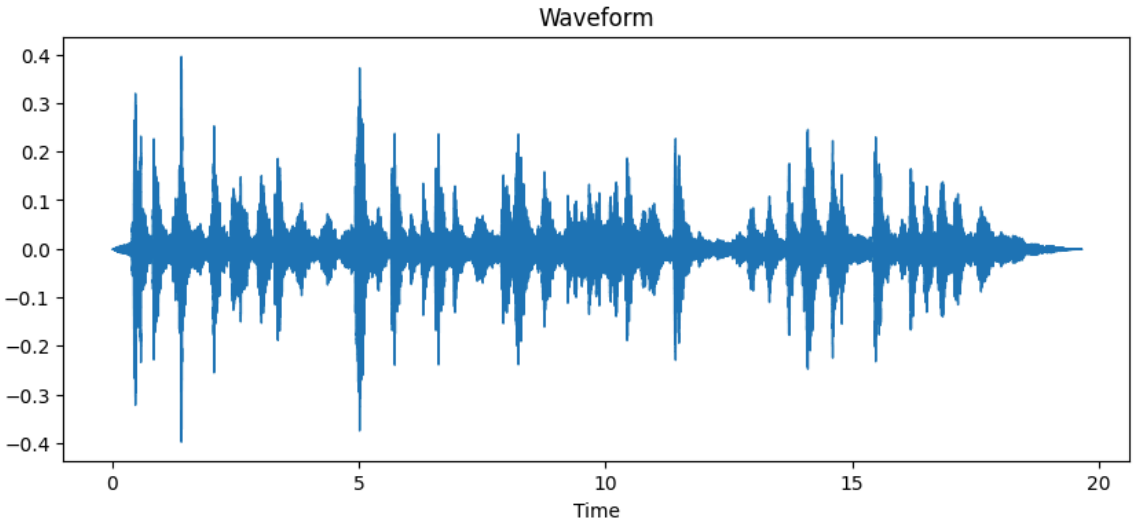
```
import os
import librosa, librosa.display
import matplotlib.pyplot as plt
from pydub import AudioSegment
from pydub.playback import pla
```

```python
# Convert MP3 -> WAV
audio = AudioSegment.from_mp3("bird_singing.mp3")
audio.export("bird_singing.wav", format="wav")

print("Playing Audio...")
play(audio)

# Load with Librosa
y, sr = librosa.load("bird_singing.wav", sr=None)

duration = len(y) / sr
size_mb = os.path.getsize("bird_singing.wav") / (1024**2)

print(f"Sample Rate: {sr} Hz")
print(f"Samples: {len(y)}")
print(f"Bits per Sample: 16")
print(f"Duration: {duration:.2f} seconds")
print(f"File Size: {size_mb:.2f} MB")
print(f"File Size per Second: {size_mb/duration:.2f} MB/s")
print(f"Max Amplitude: {max(y)}")
print(f"Min Amplitude: {min(y)}")

plt.figure(figsize=(10,4))
librosa.display.waveshow(y, sr=sr)
plt.title("Waveform")
plt.show()
```

```
Playing Audio...
Sample Rate: 44100 Hz
Samples: 866304
Bits per Sample: 16
Duration: 19.64 seconds
File Size: 3.30 MB
File Size per Second: 0.17 MB/s
Max Amplitude: 0.3968658447265625
Min Amplitude: -0.390106201171875
```

bird_singing.wav:

attachment:08e51ea4-23ee-4c19-a196-148b8f483e16:bird_singing.wav



Understanding the Output:

Property	Description
Sample Rate	Number of samples recorded per second
Samples	Total audio samples
Duration	Length of the recording
File Size	Storage required by the file
Amplitude	Signal strength of the audio
Waveform	Visual representation of the sound over time

The waveform allows us to observe how loudness changes throughout the recording.

2.2. Audio Filtering

Once an audio signal is loaded, we can modify it in various ways. Filtering allows us to remove unwanted frequencies and isolate specific parts of a sound.

This technique is heavily used in: Music production; Voice enhancement; Noise reduction; Telecommunications; Audio restoration

To demonstrate filtering, we will first generate three synthetic sounds:

- A sine wave (440 Hz)
- A square wave (330 Hz)
- White noise

We then combine them, adjust their volume, and apply several filtering techniques.

```
from pydub import AudioSegment
from pydub.generators import Sine, Square, WhiteNoise
from pydub.playback import play
import numpy as np
from scipy.signal import butter, lfilter

# Generate signals
sine = Sine(440).to_audio_segment(duration=2000)
square = Square(330).to_audio_segment(duration=2000)
noise = WhiteNoise().to_audio_segment(duration=2000)

for name, sound in [("Sine", sine), ("Square", square), ("N
oise", noise)]:
    print(name, sound.duration_seconds, "seconds")
    play(sound)

# Combine sounds
combined = sine + square + noise
combined.export("combined.wav", format="wav")

print("Playing Combined Signal...")
play(combined)

# Volume adjustments
(combined + 20).export("louder.wav", format="wav")
((sine * 2) + (square * 2) + (noise * 2)).export("looped.wa
v", format="wav")

# Built-in filters
low_pass = combined.low_pass_filter(1000)
high_pass = combined.high_pass_filter(1000)

play(low_pass)
play(high_pass)

low_pass.export("low_pass.wav", format="wav")
high_pass.export("high_pass.wav", format="wav")
```

```
# NumPy + SciPy filtering
samples = np.array(combined.get_array_of_samples())

def lowpass(data, cutoff, fs):
    b, a = butter(5, cutoff/(0.5*fs), btype="low")
    return lfilter(b, a, data)

filtered = lowpass(samples, 1000, combined.frame_rate)

filtered_audio = AudioSegment(filtered.astype(np.int16).tob
ytes(),
                              frame_rate=combined.frame_rat
e,
                              sample_width=2,
                              channels=1)

play(filtered_audio)
filtered_audio.export("filtered_output.wav", format="wav")
```

```
Sine 2.0 seconds
Square 2.0 seconds
Noise 2.0 seconds
Playing Combined Signal...
<_io.BufferedRandom name='filtered_output.wav'>
```

combined.wav:

attachment:6b4325da-be19-453d-94f7-07e3d599be8a:combined.wav

louder.wav:

attachment:7f26c255-f7a2-4d95-b3be-44bf5f1a3524:louder.wav

looped.wav:

attachment:917115d3-e8b4-4bec-8b49-967acf21d3a9:looped.wav

low_pass.wav:

attachment:428e3049-6fc8-40b3-a819-ceb8f8ef69af:low_pass.wav

high_pass.wav:

attachment:0f8ac48f-3693-470d-a815-2e64fc742318:high_pass.wav

filtered_output.wav:

attachment:60feac40-4d42-41f6-ba37-2c1647f84c70:filtered_output.wav

Common Audio Filters

Filter	Purpose
Low-Pass Filter	Keeps low frequencies, removes high frequencies
High-Pass Filter	Keeps high frequencies, removes low frequencies

Filter	Purpose
Band-Pass Filter	Keeps frequencies within a specific range
Band-Stop Filter	Removes frequencies within a specific range
Noise Filter	Reduces unwanted background noise

Why Two Filtering Approaches? PyDub provides convenient built-in filters:

```
audio.low_pass_filter(...)
audio.high_pass_filter(...)
```

These are quick and easy to use. SciPy, however, gives much more control over filter design and is commonly used in signal processing research and advanced audio applications.

As a result: **PyDub** is excellent for everyday audio editing.; **SciPy + NumPy** are better suited for precise signal-processing tasks.

In real-world projects, both approaches are frequently used together.

2.3. Text-To-Speech

One of the most interesting applications of audio processing is **Text-To-Speech (TTS)**. Instead of analyzing existing audio, we generate audio directly from text.

Modern TTS systems can transform written sentences into natural-sounding speech and are widely used in: Voice Assistants; Accessibility Tools; Audiobooks; Language Learning Applications; Navigation Systems; Customer Support Chatbots

In Python, one of the easiest ways to perform TTS is using the **Google Text-To-Speech (gTTS)** library. In this example, we will:

1. Accept text input from the user.
2. Detect its language automatically.
3. Tokenize the text.
4. Convert the text into speech.

5. Save the generated speech as an MP3 file.
6. Analyze the generated audio using Librosa.
7. Visualize: Waveform; Mel Spectrogram; Pitch Contour

Required Libraries

```
pip install gtts nltk underthesea langdetect librosa pydub
```

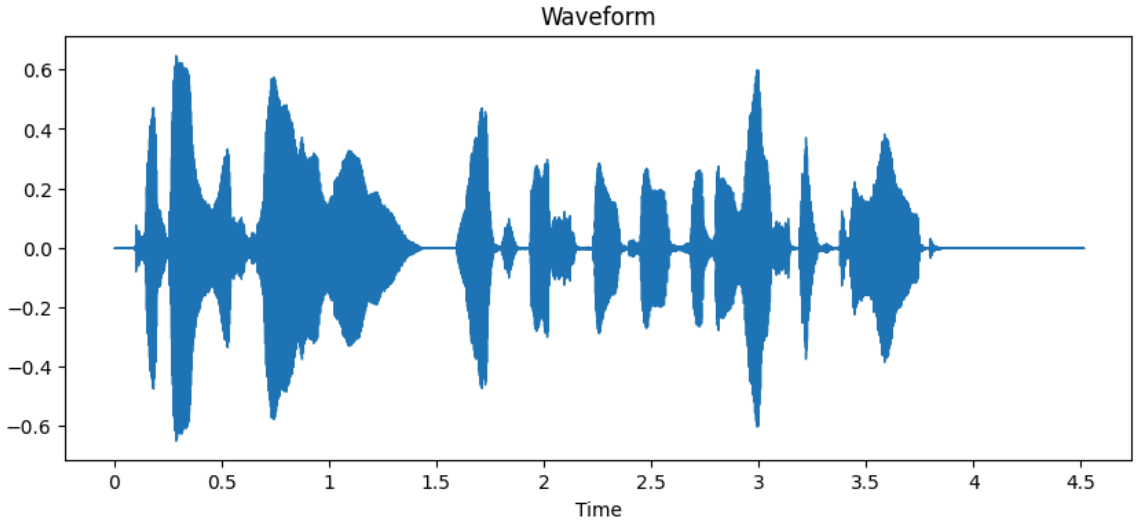
Generating Speech and Analyzing It

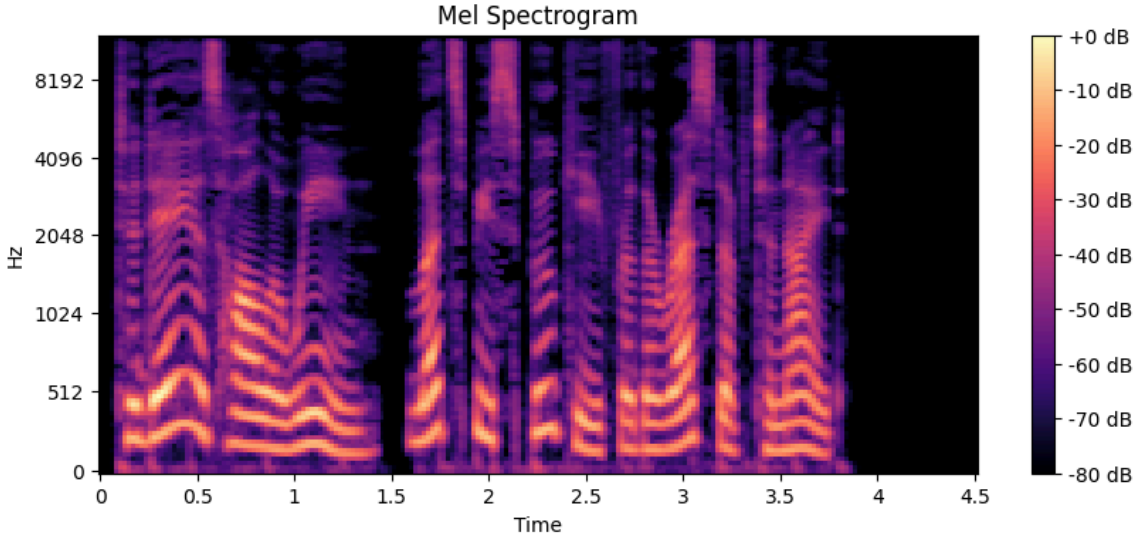
```
import nltk
from nltk.tokenize import word_tokenize
from underthesea import word_tokenize as vi_tokenize
from langdetect import detect
from gtts import gTTS
from pydub import AudioSegment
import librosa, librosa.display
import matplotlib.pyplot as plt
import numpy as np


nltk.download('punkt_tab')

text = input("Enter text: ")

lang = detect(text)
tokens = vi_tokenize(text) if lang == "vi" else word_tokeni
ze(text)

print("Tokens:", tokens)

# Generate speech
gTTS(text=text, lang=lang).save("output.mp3")

# Load audio
y, sr = librosa.load("output.mp3", sr=None)

print(f"Sample Rate: {sr}")
print(f"Duration: {len(y)/sr:.2f} seconds")
print(f"Max Amplitude: {max(y)}")
print(f"Min Amplitude: {min(y)}")

# Waveform
plt.figure(figsize=(10,4))
librosa.display.waveshow(y, sr=sr)
plt.title("Waveform")
plt.show()
```

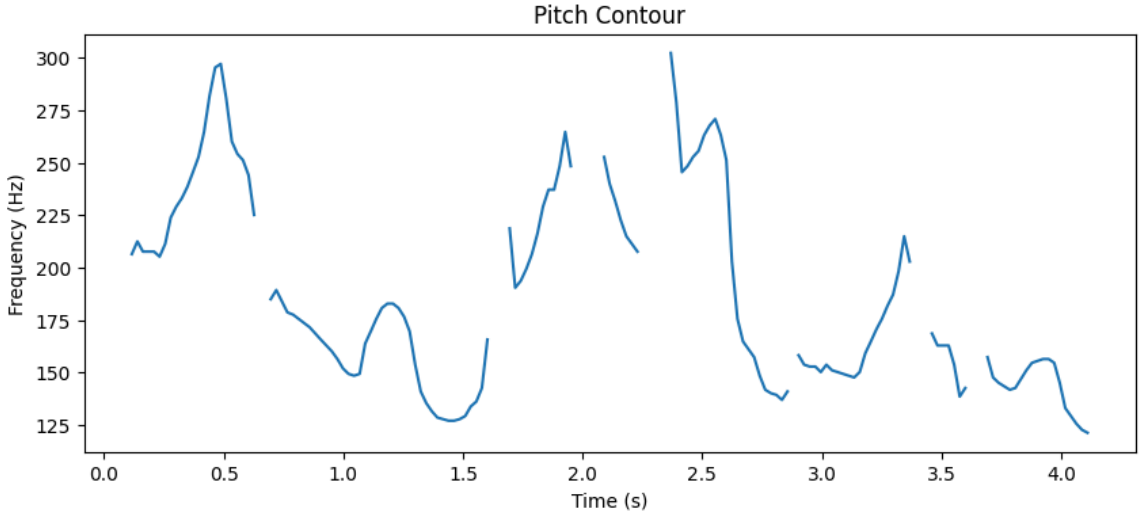
```
# Mel Spectrogram
mel = librosa.feature.melspectrogram(y=y, sr=sr)
mel_db = librosa.power_to_db(mel, ref=np.max)

plt.figure(figsize=(10,4))
librosa.display.specshow(mel_db, sr=sr, x_axis="time", y_ax
is="mel")
plt.colorbar(format="%+2.0f dB")
plt.title("Mel Spectrogram")
plt.show()

# Pitch Estimation
f0, _, _ = librosa.pyin(y, fmin=librosa.note_to_hz("C2"),
                        fmax=librosa.note_to_hz("C7"))

plt.figure(figsize=(10,4))
plt.plot(librosa.times_like(f0), f0)
plt.title("Pitch Contour")
plt.xlabel("Time (s)")
plt.ylabel("Frequency (Hz)")
plt.show()
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.

Enter text: Today is Halloween, let's be spooky for the rest of tonight
Tokens: ['Today', 'is', 'Halloween', ',', 'let', "'s", 'be', 'spooky', 'for', 'the', 'rest', 'of', 'tonight']
Sample Rate: 24000
Duration: 4.51 seconds
Max Amplitude: 0.6139451265335083
Min Amplitude: -0.6490089297294617
```

attachment:70540345-5c38-4f5d-90b0-115fb7bc888e:output.mp3


Waveform
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
Time


Mel Spectrogram
8192
4096
2048
1024
512
0
Hz
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
Time
+0 dB
-10 dB
-20 dB
-30 dB
-40 dB
-50 dB
-60 dB
-70 dB
-80 dB


Pitch Contour
300
275
250
225
200
175
150
125
Frequency (Hz)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Time (s)

Understanding the Visualizations

- The waveform displays how the audio signal changes over time.
  - Large peaks indicate louder sounds.
  - Small peaks indicate quieter sounds.
  - Silence appears close to zero amplitude.
  - This is usually the first visualization used when inspecting an audio signal.
- Mel Spectrogram
  - A spectrogram shows how frequencies change throughout time.
  - The Mel Spectrogram is especially important because it mimics how humans perceive sound frequencies.
  - It is widely used in: Speech Recognition; Speaker Identification; Voice Cloning; Deep Learning for Audio
  - Brighter regions indicate stronger frequencies.
- Pitch Contour
  - Pitch represents the perceived frequency of a voice.
  - Using `librosa.pyin()`, we estimate the fundamental frequency (**F0**) throughout the recording.
  - This allows us to observe rising intonation, falling intonation, speech rhythm, and vocal characteristics
  - Pitch tracking is commonly used in: Singing Analysis; Voice Conversion; Emotion Detection; Speech Processing
- **Why Tokenization?** Before generating speech, the text is tokenized.
  - For example: `"Hello world!"` becomes `["Hello", "world", "!"]` , while Vietnamese text may become: `"Tôi yêu Python" → ["Tôi", "yêu", "Python"]`
  - Tokenization allows us to inspect and preprocess text before converting it into speech.

Summary

Audio processing is one of the most fascinating areas of Python because it combines signal processing, data analysis, visualization, and artificial intelligence into a single workflow.

In this section, we learned how to:

- Load and play audio files using PyDub
- Convert between audio formats
- Extract metadata such as duration and sample rate
- Visualize waveforms with Librosa
- Generate synthetic audio signals
- Combine and manipulate sounds
- Apply low-pass and high-pass filters
- Build custom filters using NumPy and SciPy
- Convert text into speech using gTTS
- Analyze generated speech through spectrograms and pitch estimation

Together, Librosa and PyDub provide a powerful toolkit for working with audio. Whether you are building a music analyzer, speech recognition system, voice assistant, podcast editor, or machine learning application, Python offers all the tools needed to process audio efficiently and effectively.

References

https://librosa.org/doc/latest/index.html

https://medium.com/top-python-libraries/audio-processing-with-pydub-and-librosa-enhancing-podcasts-like-a-pro-b46792130af7

https://www.youtube.com/watch?v=B31RiiRt_TE3

Next Section:

</> 35. Python: Graphviz